Laporan Tugas Besar

IF2224 Teori Bahasa Formal dan Otomata

Milestone 1- Lexical Analysis

Semester II tahun 2025/2026

Oleh:

Kelompok ReguExceptional:

Wafiq Hibban Robbany	13524016
Muhammad Nafis Habibi	13524018
An-Dafa Anza Avansyah	13524038
Wildan Abdurrahman Ghazali	13524054

Laboratorium Ilmu dan Rekayasa Komputasi

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2 April 2026

Daftar Isi

Daftar Isi **2**
Dasar Teori **3**
Perancangan dan Implementasi **4**
Pengujian **5**
Kesimpulan dan Saran **6**
Lampiran **7**

Dasar Teori

A. Analisis Leksikal

Analisis leksikal (*lexical analysis*) adalah tahap pertama dalam proses kompilasi atau interpretasi bahasa pemrograman. Pada tahap ini, compiler membaca kode sumber mentah yang pada dasarnya hanyalah rangkaian karakter dan mengubahnya menjadi rangkaian satuan makna yang disebut token. Token merupakan unit terkecil yang memiliki arti dalam sebuah program, misalnya kata kunci (*keyword*), nama variabel (*identifier*), operator, angka, atau tanda baca.

Tujuan utama dari analisis leksikal adalah untuk mempermudah tahap berikutnya, yaitu parsing. Alih-alih berurusan langsung dengan karakter mentah, tahap parsing bekerja dengan kumpulan token yang sudah terstruktur, sehingga lebih mudah memahami susunan logis dari program. Pada proses ini, analisis leksikal juga membuang bagian yang tidak diperlukan seperti spasi, tab, dan komentar, serta dapat mendeteksi kesalahan seperti simbol yang tidak dikenal sejak awal.

Proses ini dilakukan oleh komponen yang disebut *lexer*. *Lexer* membaca kode sumber dari kiri ke kanan dan menggunakan pola tertentu untuk mengenali jenis token yang berbeda-beda. Ketika menemukan urutan karakter yang cocok dengan salah satu pola tersebut, lexer membuat sebuah token yang terdiri dari jenis token dan nilai token. Contoh hasil token:

- IDENT (a)
- BECOMES
- INTCON (5)

Agar proses pembacaan tersebut dapat dijalankan secara otomatis oleh komputer, *lexer* harus diimplementasikan ke dalam bentuk *Finite Automata*, khususnya *Deterministic Finite Automata* (DFA).

B. Deterministic Finite Automata (DFA)

Salah satu perbedaan penting di antara kelas *finite automata* adalah apakah kontrol itu termasuk "deterministik," yang berarti bahwa suatu automata tidak dapat berada dalam lebih dari satu keadaan pada satu waktu atau "non-deterministik" yang artinya bahwa ia mungkin berada dalam beberapa keadaan sekaligus.

Secara formal, sebuah *Deterministic Finite Automata* (DFA) didefinisikan sebagai *quintuple* matematika, yang umumnya dinotasikan dengan persamaan berikut:

$$A = (Q, \Sigma, \square, q_0, F)$$

Dengan rincian:

- Himpunan *state* : Q
- Himpunan karakter untuk input (alfabet) : Σ
- Fungsi transisi : $\delta(q \in Q, a \in \Sigma) \rightarrow p \in Q$
- *Start state* : $q_0 \in Q$
- Himpunan *final state* : $F \subseteq Q$

Sifat deterministik pada automata ini menegaskan bahwa pada suatu state tertentu, untuk setiap simbol input spesifik yang diberikan, hanya terdapat tepat satu state tujuan yang mungkin dilalui. Dengan demikian, mesin ini beroperasi tanpa adanya ambiguitas

Mekanisme kerja mesin ini bersifat sistematis dan terstruktur. Proses dimulai dengan inisialisasi pada *start state* (q_0). Selanjutnya, seiring mesin membaca urutan karakter input, fungsi transisi (δ) akan mengarahkan perpindahan *state* secara beruntun. Apabila setelah seluruh karakter input selesai diproses mesin berhenti tepat di salah satu *state* yang merupakan anggota dari himpunan *final state* (F), maka urutan karakter tersebut dinyatakan diterima. Sebaliknya, jika mesin berhenti di luar *final state* atau terjebak dalam *dead state*, maka input dinyatakan ditolak.

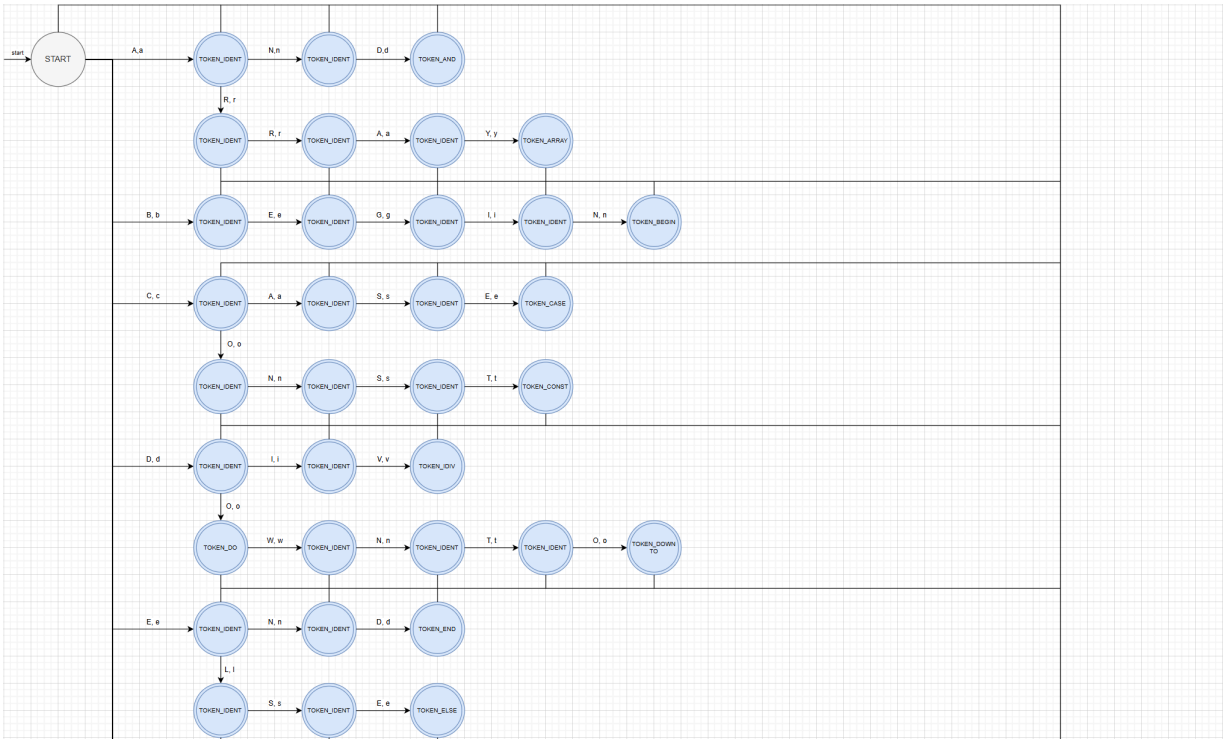
Tentunya menentukan DFA sebagai lima *tuple* dengan deskripsi rinci dari fungsi transisi sangat melelahkan dan sulit untuk dibaca. Ada dua notasi yang disarankan untuk mendeskripsikan automata yaitu sebuah diagram transisi dan tabel transisi yakni daftar tabular dari fungsi transisi yang secara implisit memberitahu kita himpunan *states* dan alfabet input.

Perancangan dan Implementasi

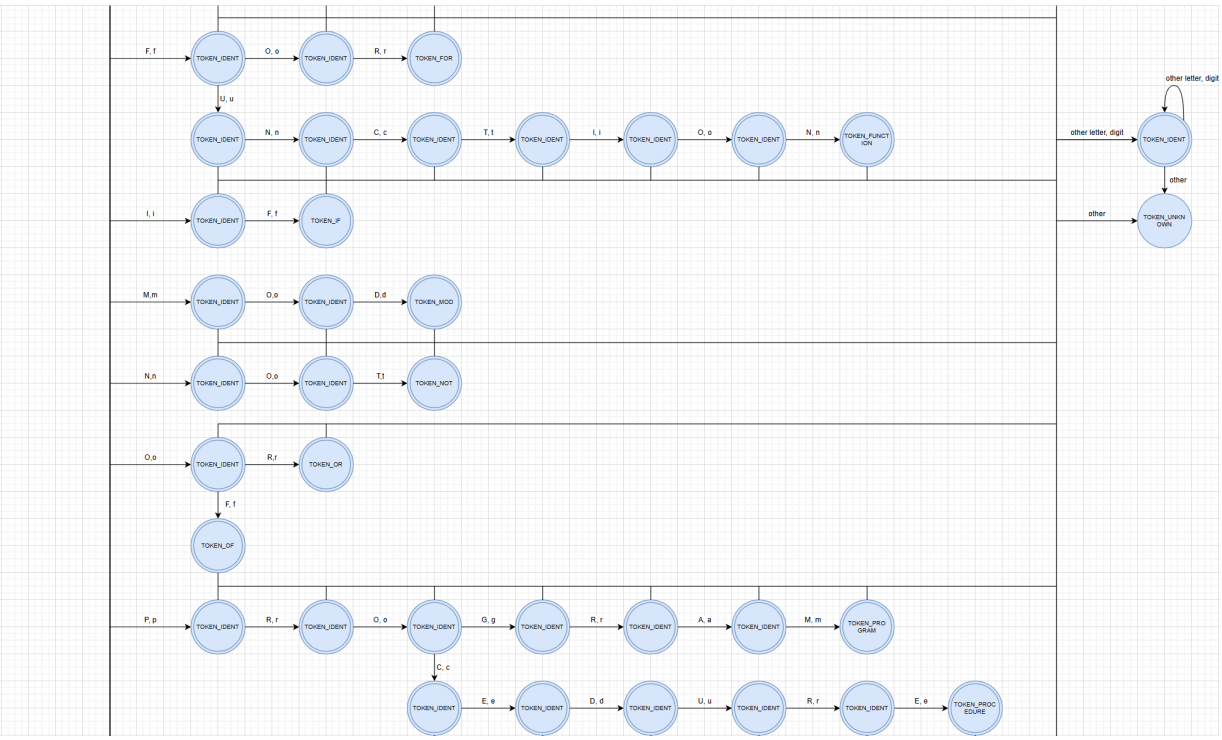
A. Perancangan Diagram Transisi DFA

Bagian ini menguraikan perancangan *Deterministic Finite Automata* (DFA) yang digunakan untuk mengenali seluruh token dalam bahasa pemrograman Arion. Mengingat bahasa Arion memiliki 52 jenis token yang berbeda, diagram DFA secara keseluruhan dipecah menjadi 6 sub-sistem berdasarkan kategori leksikalnya agar lebih terstruktur dan mudah dipahami. Dalam perancangan ini, seluruh proses pembacaan karakter selalu diinisialisasi dari sebuah *Start State* utama (State 0). Mesin akan membaca karakter per karakter dari *source code* dan berpindah *state* hingga menemukan *final state* yang mengonfirmasi pembentukan suatu token.

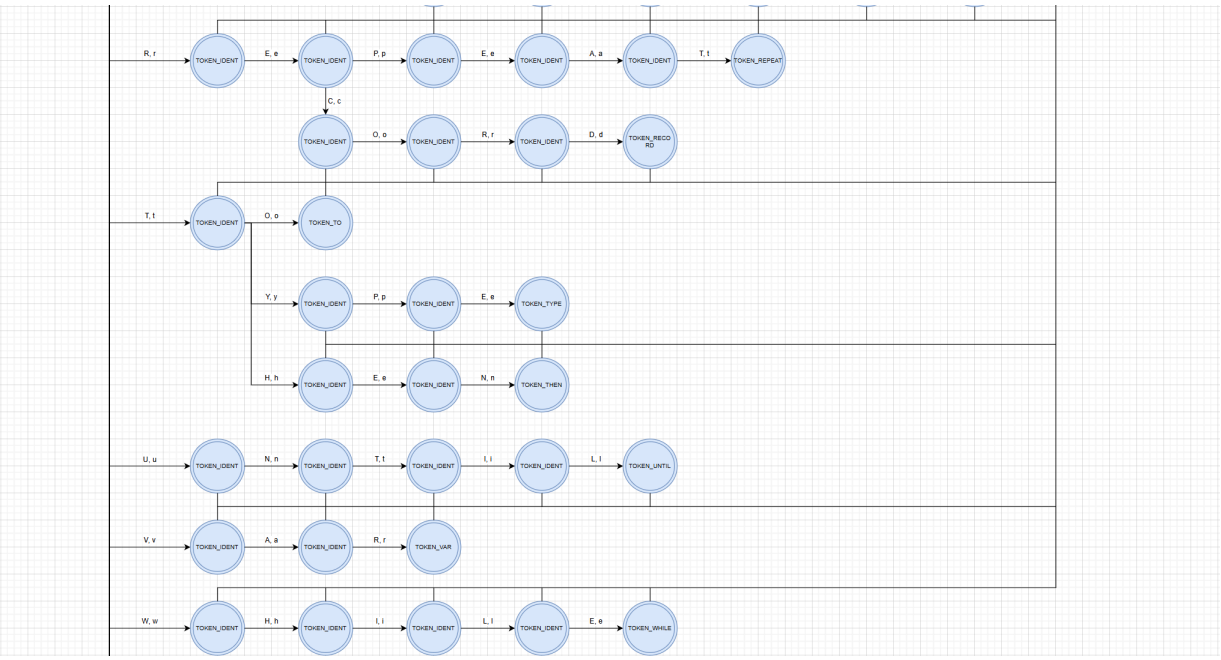
1. Pengenalan Identifier dan Keyword



Gambar 1. Diagram Transisi DFA untuk Pengenalan *Identifier* dan *Keyword (bag. 1)*



Gambar 2. Diagram Transisi DFA untuk Pengenalan *Identifier* dan *Keyword (bag. 2)*



Gambar 3. Diagram Transisi DFA untuk Pengenalan *Identifier* dan *Keyword (bag. 3)*

Daftar Token Identifier dan Keyword			
No	Token	Definisi & Keterangan	Contoh Pemakaian
1	**constsy**	Deklarasi konstanta	const
2	**typesy**	Deklarasi tipe data	type
3	**varsy**	Deklarasi variabel	var
4	**functionsy**	Deklarasi fungsi	function
5	**proceduresy**	Deklarasi prosedur	procedure
6	**arraysy**	Deklarasi array	array
7	**recordsy**	Deklarasi record	record
8	**programsy**	Deklarasi program	program
9	**ident**	Nama yang menunjukkan konstanta, tipe, variabel, prosedur, dan fungsi. Nama tersebut harus diawali dengan huruf, yang dapat diikuti oleh kombinasi huruf dan angka apa pun, dan berjumlah berapa pun. Bersifat case-insensitive	x, PI, MyInt, Integer, Real, Char, String h4g, identIniSangatPanjang24t etapiTetapValid
10	**beginsy**	-	begin
11	**ifsy**	-	if
12	**casesy**	-	case
13	**repeatsy**	-	repeat
14	**whilesy**	-	while
15	**forsy**	-	for
16	**endsy**	-	end
17	**elsesy**	-	else
18	**untilsy**	-	until

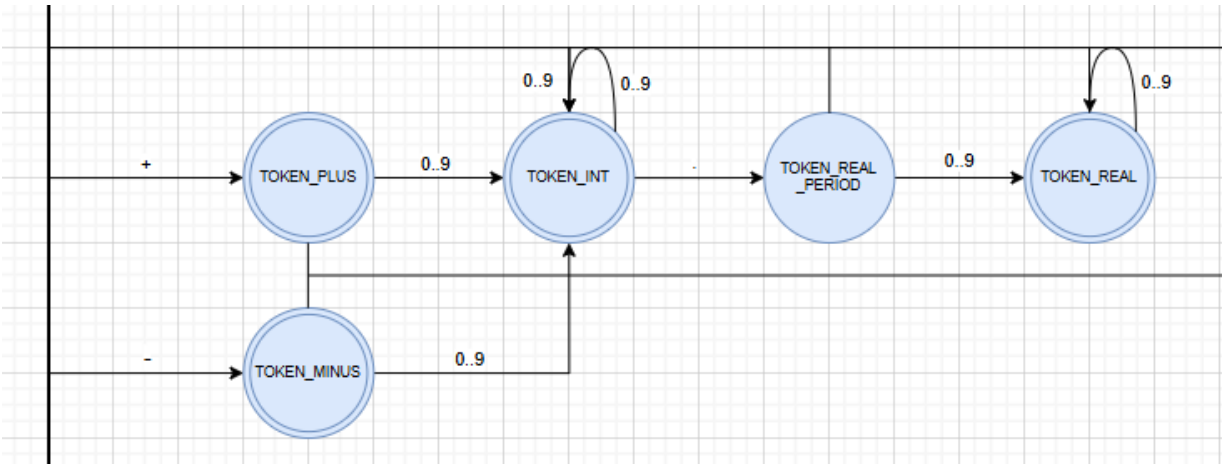
19	**ofsy**	-	of
20	**dosy**	-	do
21	**tosy**	-	to
22	**downtosy**	-	downto
23	**thensy**	-	then

Perancangan DFA pada kategori ini memetakan ejaan *keyword* secara eksplisit langkah demi langkah sejak *start state*. Karena *identifier* dan *keyword* sama-sama diawali huruf, setiap input awalan huruf akan membawa mesin ke *state* TOKEN_IDENT sekaligus menelusuri jalur kandidat *keyword* yang cocok. Selama input karakter selanjutnya terus mengikuti ejaan *keyword*, mesin akan bergerak menyusuri jalur tersebut hingga mencapai *state* akhir *keyword*. Jika pembacaan langsung diakhiri oleh karakter pemisah leksikal, maka token disahkan sebagai *keyword*.

Namun, mesin dirancang dinamis untuk membatalkan kandidat *keyword* jika terjadi penyimpangan ejaan. Apabila di tengah jalur *keyword* input menyimpang, atau terdapat tambahan huruf/angka setelah *state* akhir *keyword* tercapai, transisi akan langsung dialihkan ke *state looping* TOKEN_IDENT dan nantinya disahkan sebagai token ident. Apabila pada proses penelusuran ini mesin membaca simbol ilegal (selain huruf dan angka), transisi akan langsung dilempar ke *dead state* TOKEN_UNKNOWN, yang menandakan sekuens karakter tersebut ditolak.

2. Pengenalan Angka Integer dan Real

Daftar Token Numerik			
No	Token	Definisi & Keterangan	Contoh Pemakaian
1	**intcon**	Konstanta integer	1, 3, 48
2	**realcon**	Konstanta bilangan riil	3.14, 26.7



Gambar 4. Diagram Transisi DFA untuk Pengenalan Angka Numerik

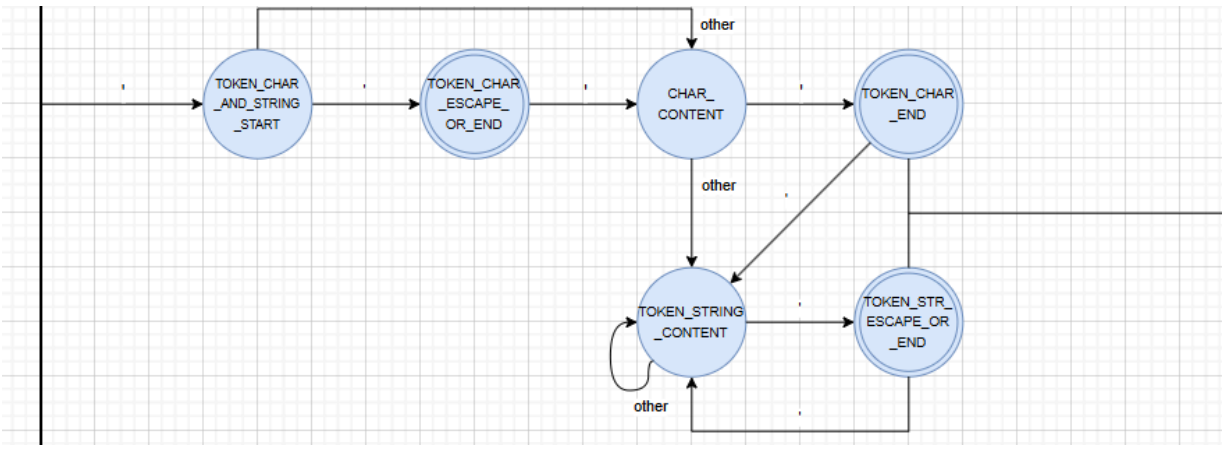
Perancangan DFA untuk angka diawali ketika mesin membaca input digit [0-9] pertama dari *start state*, yang memicu transisi menuju *state* TOKEN_INT. Selama karakter selanjutnya tetap berupa digit, mesin akan terus melakukan *looping* untuk mempertahankan kedudukannya pada *state* tersebut hingga pembacaan diakhiri oleh karakter pemisah leksikal, lalu sekuens tersebut disahkan sebagai token intcon.

Namun, jika di tengah proses mesin mendeteksi karakter titik (.), jalur transisi akan langsung dialihkan untuk mengevaluasi bilangan real, yang mewajibkan hadirnya minimal satu digit angka di belakang koma. Begitu digit desimal pertama dibaca, mesin bertransisi ke *state* TOKEN_REAL dan kembali melakukan *looping* untuk menampung digit-digit berikutnya hingga pembacaan selesai, sehingga token disahkan sebagai realcon. Apabila dalam proses perangkaian angka ini mesin menerima input huruf atau simbol ilegal, transisi akan seketika dilempar menuju *dead state* TOKEN_UNKNOWN dan deretan karakter tersebut ditolak.

3. Pengenalan String dan Karakter

Daftar Token String dan Karakter			
No	Token	Definisi & Keterangan	Contoh Pemakaian
1	**charcon**	Konstanta karakter singular, diapit dengan tanda petik tunggal (‘)	‘j’,’k’,’t’

2	**string**	Sekuens karakter, diapit dengan tanda petik tunggal (‘)	‘IRK’,’TBFO’



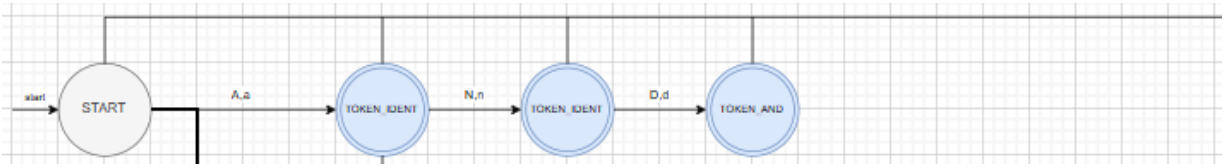
Gambar 5. Diagram Transisi DFA untuk Pengenalan String dan Karakter

Perancangan DFA pada kategori ini secara eksklusif dipicu oleh input karakter tanda petik tunggal (') dari start state. Setelah menerima kutip pembuka tersebut, mesin bertransisi ke state TOKEN_CHAR_AND_STRING_START. Dari state ini, jika karakter berikutnya bukan tanda petik, mesin bertransisi ke state CHAR_CONTENT yang akan terus melakukan looping untuk membaca karakter apa pun kecuali tanda petik tunggal. Namun jika karakter berikutnya langsung tanda petik, mesin bertransisi ke state TOKEN_CHAR_ESCAPE_OR_END sebagai evaluasi awal. Apabila mesin berada di state CHAR_CONTENT atau TOKEN_STRING_CONTENT dan menemukan karakter tanda petik, mesin bertransisi ke state evaluasi (TOKEN_CHAR_ESCAPE_OR_END atau TOKEN_STRING_ESCAPE_OR_END). Jika input berikutnya adalah tanda petik kembali, sekuens tersebut diidentifikasi sebagai escape character yang mewakili satu karakter petik literal, dan mesin bertransisi kembali ke state penampung isi. Namun jika karakter berikutnya bukan tanda petik, tanda petik sebelumnya disahkan sebagai penutup dan mesin menuju final state. Pada tahap akhir, hasil klasifikasi token ditentukan berdasarkan jalur state yang dilalui: jika mesin mencapai final state melalui

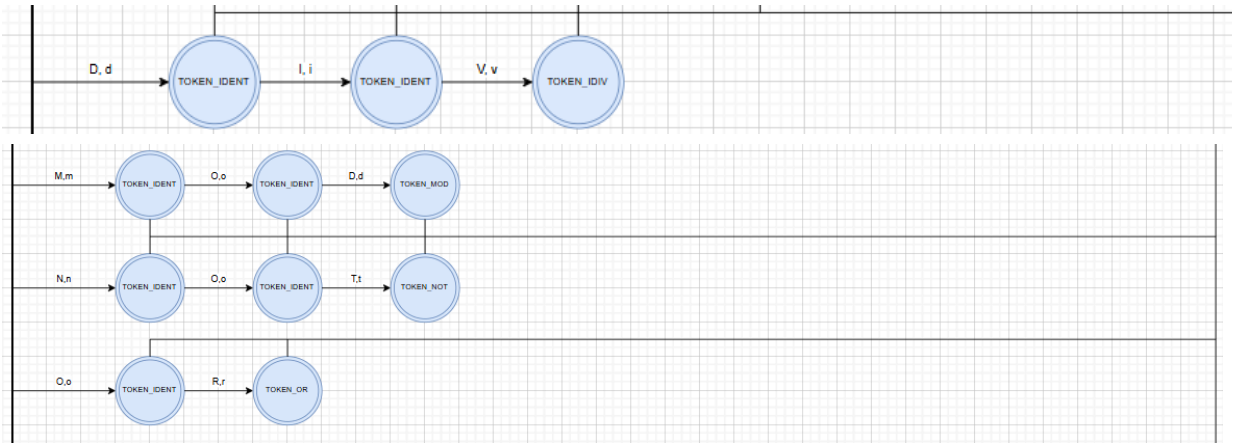
TOKEN_CHAR_END, token diklasifikasikan sebagai charcon, sedangkan jika melalui TOKEN_STRING_ESCAPE_OR_END, token diklasifikasikan sebagai string.

4. Pengenalan Operator

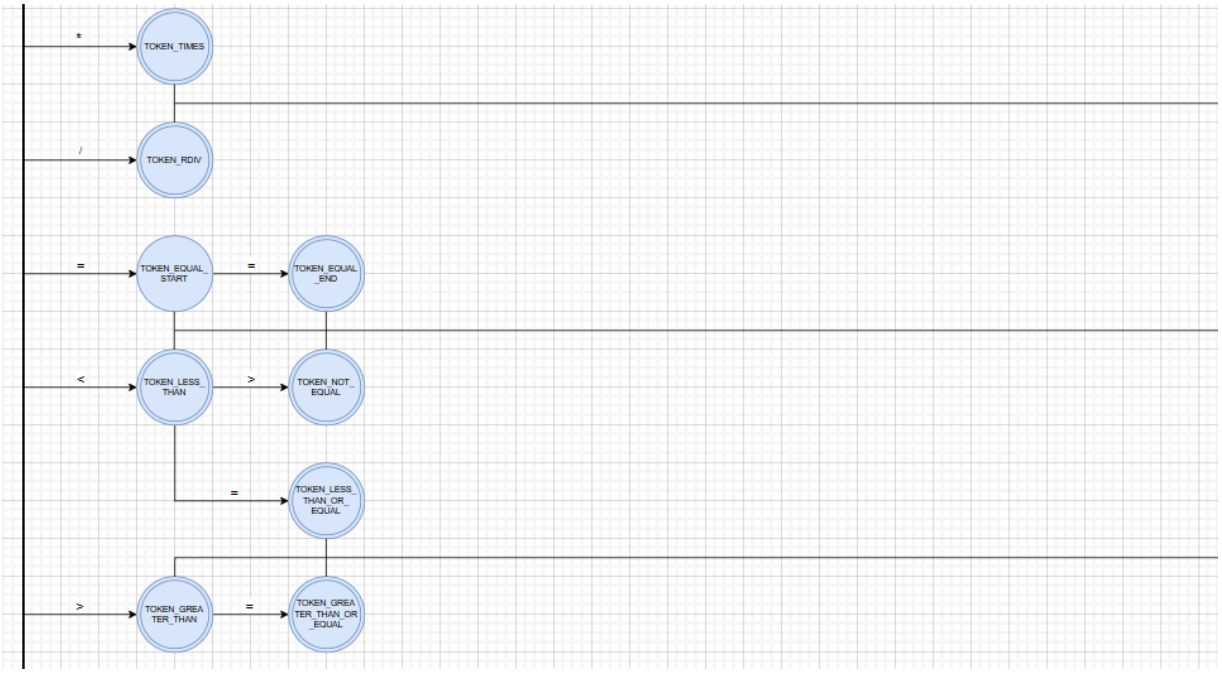
Daftar Token Operator			
No	Token	Definisi & Keterangan	Contoh Pemakaian
1	**plus**	Operator aritmatika pertambahan	+
2	**minus**	Operator aritmatika pengurangan	-
3	**times**	Operator aritmatika perkalian	*
4	**idiv**	Operator aritmatika pembagian Integer	div
5	**rdiv**	Operator aritmatika pembagian Rill	/
6	**imod**	Operator aritmatika modulo	MOD
7	**andsy**	Operator logika AND	AND
8	**orsy**	Operator logika OR	OR
9	**eql**	equal	==
10	**neq**	Not equal	<>
11	**gtr**	Greater than	>
12	**geq**	Greater than or equal	>=
13	**lss**	Less than	<
14	**leq**	Less than or equal	<=
15	**becomes**	Assignment operator	:=
16	**notsy**	Operator logika NOT (negasi)	NOT



Gambar 5. Diagram Transisi DFA untuk Operator *(bag 1)*



Gambar 6. Diagram Transisi DFA untuk Operator *(bag 2)*



Gambar 7. Diagram Transisi DFA untuk Operator *(bag 3)*

Gambar 8. Diagram Transisi DFA untuk Operator *(bag 4)*

Perancangan DFA untuk operator dapat terdiri dari bermacam-macam bentuk/jenis. Jika DFA menerima input karakter pertama untuk operator sederhana seperti plus, minus, times, rdiv, gtr, dan lss maka DFA langsung berada pada *final state* sesuai dengan token operator tersebut. Adapun operator yang terbentuk dari gabungan dua operator atau simbol yaitu seperti eql, neq, geq, leq, dan becomes yang mengharuskan DFA membaca dua input karakter untuk sampai ke *final state*. Misalkan suatu DFA membaca input pertama karakter atau simbol yang valid, maka pada input karakter kedua ia harus membaca karakter yang dapat membentuk suatu token jika digabungkan dengan input karakter pertamanya. Jika input karakter kedua tidak valid maka akan mengarah ke *dead state* yakni TOKEN_UNKNOWN. Adapun jenis operator terakhir yang dibentuk dari *letter of alphabet*, operator tersebut antara lain adalah idiv, imod, andsy, dan orsy. Cara membaca setiap token dilakukan secara sekuensial untuk input string yang diterima, misalkan saja token idiv yang mengharuskan DFA membaca karakter pertama ‘d’, karakter kedua ‘i’, dan karakter ketiga ‘v’. Perlu diingat bahwa pembaca DFA bersifat *case-insensitive* sehingga capital letter juga dapat membentuk token yang dimaksud. Jika pada tahapan membaca karakter kedua atau ketiga tidak memenuhi token idiv, maka ada kemungkinan bahwa token atau state yang dituju nantinya adalah state lain, seperti indent jika input karakternya adalah *letter of alphabet* namun tidak merepresentasikan token apapun dalam bahasa pemrograman Arion dan *dead state* atau TOKEN_UNKNOWN jika input karakter-karakter selanjutnya bukanlah *letter of alphabet* ataupun digit.

5. Pengenalan *Punctuator* dan *Separator*

Daftar Token *Punctuator* dan *Separator*			
No	Token	Definisi & Keterangan	Contoh Pemakaian
1	**lparent**	Left parentheses	(

2	**rparent**	Right parentheses	)
3	**lbrack**	Kurung siku kiri	[
4	**rbrack**	Kurung siku kanan	]
5	**comma**	-	,
6	**semicolon**	-	;
7	**period**	-	.
8	**colon**	-	:

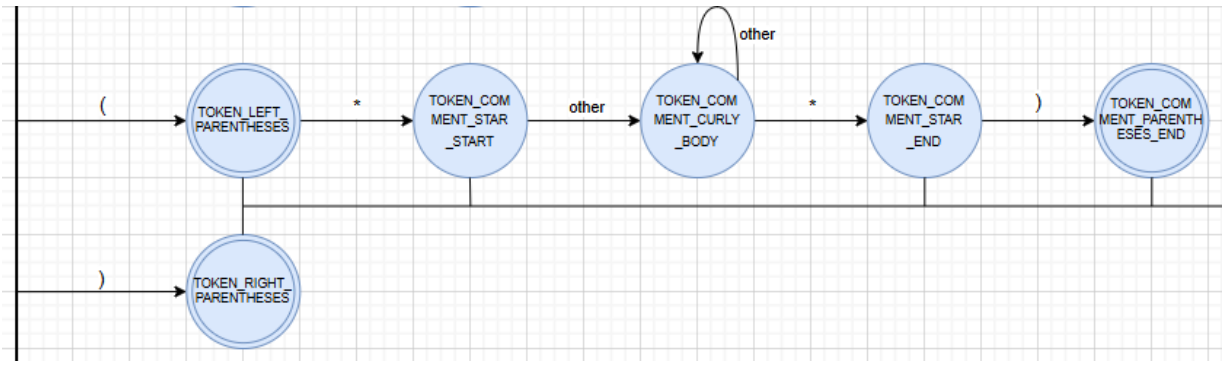
Gambar 9. Diagram Transisi DFA untuk *Punctuator* dan *Separator*

Perancangan DFA untuk membaca token *separator* maupun *punctuator* cukuplah sederhana, yakni jika DFA menerima input karakter pertama berupa simbol atau token yang terdaftar dalam daftar token *punctuator* dan *separator*, maka DFA akan memasuki

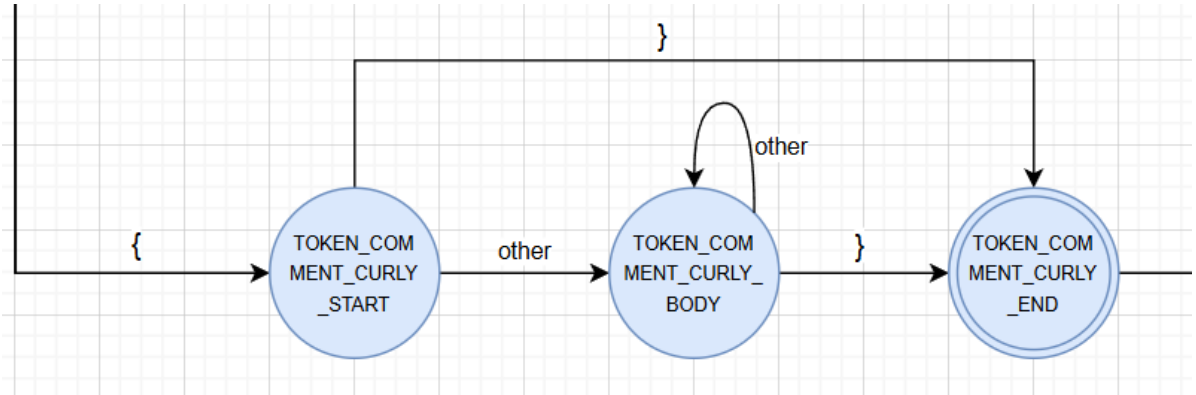
final state sesuai dengan token yang dibaca.

6. Pengenalan Komentar

Token Komentar			
No	Token	Definisi & Keterangan	Contoh Pemakaian
1	**comment**	Dimulai dengan { atau (* (bukan di dalam tanda kutip) dan diakhiri dengan } atau *).	{ini komentar}, (*Ini komentar*)



Gambar 10. Diagram Transisi DFA untuk Komentar *(bag 1)*



Gambar 11. Diagram Transisi DFA untuk Komentar *(bag 2)*

Pada perancangan DFA untuk mengenali komentar, terdapat dua mekanisme berbeda untuk setiap jenis *syntax* komentar. *Syntax* komentar yang pertama adalah menggunakan (* *), dimana DFA menerima input karakter pertama lparent, setelah itu

input karakter kedua yang dibaca DFA adalah simbol bintang (*). Setelah kedua kombinasi simbol tersebut, input apapun yang diterima DFA akan dianggap sebagai *content* dari komentar hingga DFA membaca karakter bintang dan rparent secara sekuensial. *Syntax* kedua adalah dengan menggunakan kurung kurawal {}, dimana pada input karakter pertama, DFA menerima kurung kurawal buka. Setelah itu apapun input-nya maka akan menjadi *content* dalam komentar yang kita buat hingga kita bertemu dengan kurung kurawal tutup. Jika setelah kita bertemu kurung kurawal tutup lagi setelah kurung kurawal tutup yang pertama maka program tidak valid.

B. Implementasi

Implementasi dilakukan dalam bahasa C++ dan dibuat dalam *namespace* "arion" agar terpisah dari *standard library* C++.

1. DFA

DFA dibuat dalam bentuk kelas (*class*). DFA diimplementasikan sesuai dengan definisi DFA itu sendiri yang berupa *quintuple.* Semua *state*, *start state, final state*, dan transisi harus didefinisikan terlebih dahulu sebelum DFA dijalankan.

State direpresentasikan dengan integer 32 bit sehingga banyak *state* yang mungkin dari DFA ada sekitar 4 miliar. Setiap *state* diharuskan memiliki nama untuk keperluan *debugging* dan *output*. Untuk memudahkan memasukkan *state*, *state* didefinisikan sebagai enum karena setiap nilai dari enum sebenarnya adalah bilangan bulat. Sebelum dapat digunakan, *state* yang ada pada DFA harus didefinisikan terlebih dahulu. Jika menentukan *start state* atau menambah transisi pada *state* yang belum didefinisikan, DFA akan mengeluarkan *exception*.

DFA memiliki *state* 0 sebagai *default* dari *start state*. *Start state* dapat ditentukan dari *state* yang telah ada pada DFA. *Current state* dari DFA perlu di-*reset* menjadi *start state* terlebih dahulu sebelum melakukan transisi apapun dengan input. Jika tidak, transisi yang terjadi mungkin tidak sesuai harapan.

Setelah *state* dimasukkan ke dalam DFA, *state* tersebut dapat ditandai sebagai salah satu *final state*. C*urrent state* dari DFA dapat dicek apakah merupakan salah satu dari *final state*.

Input untuk transisi berupa integer 32 bit. Hal ini memungkinkan input untuk menggunakan karakter (8 bit), enum, atau yang lainnya. Menambahkan transisi dapat dilakukan dengan menentukan dari *state* mana, dengan input apa, ke *state* mana. Tidak ada boleh ada dua transisi berbeda dari *state* dan dengan input yang sama ke *state* yang berbeda. Untuk mengganti tujuan transisi, transisi yang lama harus dihapus terlebih dahulu lalu transisi yang baru ditambahkan.

Dimulai dari *start state*, DFA dapat diberikan input untuk melakukan transisi secara otomatis. Jika input valid, DFA akan transisi ke *state* yang ditentukan. Jika input tidak valid, DFA akan transisi ke *state* "invalid" yang direpresentasikan sebagai *state* -1. Tidak ada transisi keluar dari *state* "invalid" tersebut. Untuk kembali ke *start state*, DFA harus di-*reset*.

Berikut definisi dari kelas DFA pada src/arion/DFA.hpp.

```
class DFA {
public:
    // Constant(s)
    const int INVALID_STATE = -1;

    // Constructor
    DFA();

    // Current state
    int getCurrentState() const;
    bool isCurrentStateInvalid() const;
    void resetToStartState();

    // Start state
    int getStartState() const;
    void setStartState(int stateId);

    // State
    bool hasState(int stateId) const;
    std::string getStateName(int stateId);
    void addState(int stateId, std::string name);
    void removeState(int stateId);

    // Final State
```

```
    bool isFinalState(int stateId) const;
    void setFinalState(int stateId, bool isFinalState);

    // Transition
    bool transition(int input);
    bool canTransition(int input);
    bool canTransition(int from, int input);
    void addTransition(int from, int input, int to);
    void addTransition(int from, std::string inputs, int to);
    void removeTransition(int from, int input);
    void removeTransition(int from, std::string inputs);

private:
    struct hash {
        std::size_t operator()(const std::pair<int, int> &p)
const
        {
            return ((unsigned long long)p.first << 32) ^
p.second;
        }
    };
    int currentState_ = 0;
    int startState_ = 0;
    std::unordered_map<int, std::string> states_;
    std::unordered_set<int> finalStates_;
    std::unordered_map<std::pair<int, int>, int, DFA::hash>
transitionMap_;
};
```

2. Lexer/Tokenizer

Tokenizer merupakan program untuk mendapatkan token dari sebuah program. Tokenizer memanfaatkan DFA untuk mendapatkan token dengan membaca karakter masukan satu per satu. Langkah awal dari Tokenizer adalah menginisiasi start state, kemudian memulai pembacaan karakter untuk transisi DFA. Setelah DFA mencapai final state terakhir (state yang merupakan final state dan tidak bisa transisi), maka Tokenizer akan mengembalikan token berdasarkan leksem (kata) terakhir yang dibaca, atau final state terakhir DFA.

Karakter pertama yang dibaca dari start state akan mempengaruhi bagaimana Tokenizer mendapatkan token :

a. Jika karakter yang dibaca adalah karakter abjad (a-z, *case-insensitive*), token yang didapatkan adalah token IDENT. Setelah semua karakter dibaca dan leksem (sebuah kata) didapatkan, maka untuk token-token yang didapatkan dari *keyword* (kata kunci) seperti *if, do, while*, token diambil dengan menggunakan std::map, dimana *keyword* sebagai *key* dan token sebagai *value*. Token didapatkan dengan menggunakan fungsi bawaan std::map, yaitu find(). Apabila karakter bukan token dari *keyword,* maka token adalah token IDENT.
b. Jika karakter yang dibaca bukan karakter abjad, maka token akan diambil dari final state terakhir DFA.

Tokenizer diimplementasikan dalam bentuk class yang memiliki beberapa metode berikut:

a. Token getNextToken()
   Metode yang digunakan untuk mengambil token selanjutnya dari sebuah leksem (kata) dengan menelusuri transisi state pada DFA dimulai dari start state hingga mencapai final state.
b. char getChar()
   Metode ini mengambil karakter selanjutnya untuk melakukan transisi pada DFA. Metode ini digunakan dalam implementasi Token getNextToken().
c. char peekChar()
   Metode ini melihat karakter selanjutnya untuk mengecek apakah DFA akan melakukan transisi di posisi final state. Metode ini digunakan dalam implementasi Token getNextToken().
d. void skipWhitespace()
   Metode ini mengabaikan spasi atau *tab.* Metode ini digunakan dalam implementasi Token getNextToken().
e. string toLower(const std::string&)
   Metode ini mengonversi karakter huruf abjad kapital menjadi huruf abjad kecil. Metode ini digunakan dalam implementasi Token getNextToken().

f. string tokenToString(Token type, const std::string& lexeme)

   Metode ini mengubah tipe token menjadi string seperti *if* menjadi *ifsy, while* menjadi *whilesy*. dsb. Metode ini digunakan dalam program *main*.

g. void setStream(const std::string &) dan void setStream(std::ifstream &)

   Metode ini digunakan untuk membuka input stream dari file. Digunakan dalam program *main*.

h. bool isStreamOpen()

   Metode ini digunakan untuk memastikan bahwa input stream sudah terbuka.

i. void closeStream()

   Metode ini digunakan untuk menutup input stream jika diperlukan.

j. void setDebug(bool)

   Metode ini akan mengaktifkan/menonaktifkan output debug saat melakukan tokenisasi.

Untuk atribut, Tokenizer memiliki sebuah DFA, kumpulan state untuk tiap leksem, string masukan (*input*), posisi karakter pada input, lexeme yang menyimpan string dari start state hingga final state, dan map yang memetakan *keyword* ke token.

```
class Tokenizer {

public:
    enum State {
        START,
        TOKEN_INT,
        TOKEN_REAL_PERIOD,
        TOKEN_REAL,
        TOKEN_CHAR_AND_STRING_START,
        TOKEN_CHAR_CONTENT,
        TOKEN_CHAR_END,
        TOKEN_CHAR_ESCAPE_OR_END,
        TOKEN_STRING_CONTENT,
        TOKEN_STRING_ESCAPE_CHAR,
        TOKEN_STRING_ESCAPE_OR_END,
        TOKEN_PLUS,
        TOKEN_MINUS,
        TOKEN_TIMES,
        TOKEN_RDIV,
```

```
        TOKEN_EQUAL_START,
        TOKEN_EQUAL_END,
        TOKEN_NOT_EQUAL,
        TOKEN_GREATER_THAN,
        TOKEN_GREATER_THAN_OR_EQUAL,
        TOKEN_LESS_THAN,
        TOKEN_LESS_THAN_OR_EQUAL,
        TOKEN_LEFT_PARENTHESES,
        TOKEN_RIGHT_PARENTHESES,
        TOKEN_LEFT_BRACKET,
        TOKEN_RIGHT_BRACKET,
        TOKEN_COMMA,
        TOKEN_SEMICOLON,
        TOKEN_PERIOD,
        TOKEN_COLON,
        TOKEN_BECOMES,
        TOKEN_IDENT,
        TOKEN_NOT,
        TOKEN_IDIV,
        TOKEN_MOD,
        TOKEN_AND,
        TOKEN_OR,
        TOKEN_CONST,
        TOKEN_TYPE,
        TOKEN_VAR,
        TOKEN_FUNCTION,
        TOKEN_PROCEDURE,
        TOKEN_ARRAY,
        TOKEN_RECORD,
        TOKEN_PROGRAM,
        TOKEN_BEGIN,
        TOKEN_IF,
        TOKEN_CASE,
        TOKEN_REPEAT,
        TOKEN_WHILE,
        TOKEN_FOR,
        TOKEN_END,
        TOKEN_ELSE,
        TOKEN_UNTIL,
        TOKEN_OF,
        TOKEN_DO,
        TOKEN_TO,
        TOKEN_DOWNTO,
        TOKEN_THEN,
        TOKEN_COMMENT_CURLY_START,
        TOKEN_COMMENT_PARENTHESES_START,
        TOKEN_COMMENT_STAR_START,
        TOKEN_COMMENT_CURLY_BODY,
```

```
        TOKEN_COMMENT_STAR_BODY,
        TOKEN_COMMENT_CURLY_END,
        TOKEN_COMMENT_STAR_END,
        TOKEN_COMMENT_PARENTHESES_END,
        TOKEN_EOF,
        TOKEN_UNKNOWN
    };
    Tokenizer();

    void setStream(const std::string &);
    void setStream(std::ifstream &);
    bool isStreamOpen();
    void closeStream();

    void setDebug(bool);
    Token getNextToken();
    std::string tokenToString(Token type);
    std::string getLexeme() { return lexeme_; };

private:
    char peekChar();
    char getChar();
    void skipWhitespace();
    std::string toLower(const std::string &);

    bool debug_ = false;
    DFA dfa_;
    std::ifstream input_;
    std::string lexeme_;
    std::unordered_map<std::string, int> wordTokens_ = {
        {"not", TOKEN_NOT},
        {"div", TOKEN_IDIV},
        {"mod", TOKEN_MOD},
        {"and", TOKEN_AND},
        {"or", TOKEN_OR},
        {"const", TOKEN_CONST},
        {"type", TOKEN_TYPE},
        {"var", TOKEN_VAR},
        {"function", TOKEN_FUNCTION},
        {"procedure", TOKEN_PROCEDURE},
        {"array", TOKEN_ARRAY},
        {"record", TOKEN_RECORD},
        {"program", TOKEN_PROGRAM},
        {"begin", TOKEN_BEGIN},
        {"if", TOKEN_IF},
        {"case", TOKEN_CASE},
        {"repeat", TOKEN_REPEAT},
        {"while", TOKEN_WHILE},
```

```
        {"for", TOKEN_FOR},
        {"end", TOKEN_END},
        {"else", TOKEN_ELSE},
        {"until", TOKEN_UNTIL},
        {"of", TOKEN_OF},
        {"do", TOKEN_DO},
        {"to", TOKEN_TO},
        {"downto", TOKEN_DOWNTO},
        {"then", TOKEN_THEN}};
};
```

Pengujian

Kasus uji dapat dilihat di bagan berikut dan folder test/milestone-1/ di repositori GitHub

No.	Input file dan isi	Output file dan isi

1.	input-1.txt
	test > milestone-1 > input-1.txt
	`program Hello;`
	
	`var`
	`  a, b: integer;`
	
	`begin`
	`  a := { Hello Saya Nafis }  5;`
	`  b := a + 10;`
	`  writeln('Result = ', b);`
	`end.`
		tokenized-input-1.txt
		test > milestone-1 > tokenized-input-1.txt
		`programsy`
		`ident (Hello)`
		`semicolon`
		`varsy`
		`ident (a)`
		`comma`
		`ident (b)`
		`colon`
		`ident (integer)`
		`semicolon`
		`beginsy`
		`ident (a)`
		`becomes`
		`comment( Hello Saya Nafis )`
		`intcon (5)`
		`semicolon`
		`ident (b)`
		`becomes`
		`ident (a)`
		`plus`
		`intcon (10)`
		`semicolon`
		`ident (writeln)`
		`lparent`
		`string ('Result = ')`
		`comma`
		`ident (b)`
		`rparent`
		`semicolon`
		`endsy`
		`period`

2.	input-2.txt test > milestone-1 > input-2.txt const type var function procedure array record begin if case repeat while for end else until of do to downto then	Tokenized-input-2.txt test > milestone-1 > tokenized-input-2.txt constsy typesy varsy functionsy proceduresy arraysy recordsy beginsy ifsy casesy repeatsy whilesy forsy endsy elsesy untilsy ofsy dosy tosy downtosy thensy

3.	input-3.txt
	test > milestone-1 > input-3.txt
	begin
	a:=b+c*d-e/f;
	end.
		tokenized-input-3.txt
		test > milestone-1 > tokenized-input-3.txt
		beginsy
		ident (a)
		becomes
		ident (b)
		plus
		ident (c)
		times
		ident (d)
		minus
		ident (e)
		rdiv
		ident (f)
		semicolon
		endsy
		period

4.	input-4.txt
	test > milestone-1 > input-4.txt
	`begin`
	`  a:=b<=c>=d<>e< f >g;`
	`end.`
		tokenized-input-4.txt
		test > milestone-1 > tokenized-input-4.txt
		`beginsy`
		`ident (a)`
		`becomes`
		`ident (b)`
		`leq`
		`ident (c)`
		`geq`
		`ident (d)`
		`neq`
		`ident (e)`
		`lss`
		`ident (f)`
		`gtr`
		`ident (g)`
		`semicolon`
		`endsy`
		`period`

5.	input-5.txt
	test > milestone-1 > input-5.txt
	`program programbegin beginx endy var1`
		tokenized-input-5.txt
		test > milestone-1 > tokenized-input-5.txt
		`programsy`
		`ident (programbegin)`
		`ident (beginx)`
		`ident (endy)`
		`ident (var1)`

6.	input-6.txt
	test > milestone-1 > input-6.txt
	`{ "Lorem ipsum dolor sit amet,`
	`  consectetur adipiscing elit,`
	`  sed do eiusmod tempor incididunt`
	`  ut labore et dolore magna aliqua.`
	`  Ut enim ad minim veniam,`
	`  quis nostrud exercitationullamco`
	`  laboris nisi ut aliquip ex ea commodo consequat.`
	`  Duis aute irure dolor in reprehenderit in`
	`  voluptate velit esse cillum dolore eu fugiat nulla pariatur.`
	`  Excepteur sint occaecat cupidatat non proident,`
	`  sunt in culpa qui officia deserunt mollit anim id est laborum."`
	`(* {** (* Ya ini komentar *) *) }`
	`_snake_case_`
	`kebab-case`
	`camelCase`
	`PascalCase`
		tokenized-input-6.txt
		test > milestone-1 > tokenized-input-6.txt
		`comment( "Lorem ipsum dolor sit amet,`
		`  consectetur adipiscing elit,`
		`  sed do eiusmod tempor incididunt`
		`  ut labore et dolore magna aliqua.`
		`  Ut enim ad minim veniam,`
		`  quis nostrud exercitationullamco`
		`  laboris nisi ut aliquip ex ea commodo consequat.`
		`  Duis aute irure dolor in reprehenderit in`
		`  voluptate velit esse cillum dolore eu fugiat nulla pariatur.`
		`  Excepteur sint occaecat cupidatat non proident,`
		`  sunt in culpa qui officia deserunt mollit anim id est laborum." )`
		`comment( {** (* Ya ini komentar )`
		`times`
		`rparent`
		`unknown ({})`
		`unknown (_)`
		`ident (snake)`
		`unknown (_)`
		`casesy`
		`unknown (_)`
		`ident (kebab)`
		`minus`
		`casesy`
		`ident (camelCase)`
		`ident (PascalCase)`

7.	input-7.txt test > milestone-1 > input-7.txt '(* AKU *)' '{----------)' 'Kalau sampai waktuku' '''Ku mau tak seorang ''kan merayu' 'Tidak juga kau' 'Tak perlu sedu sedan itu' 'Aku ini binatang jalang' 'Dari kumpulannya terbuang' 'Biar peluru menembus kulitku' 'Aku tetap meradang menerjang' 'Luka dan bisa kubawa berlari' 'Berlari' 'Hingga hilang pedih peri' 'Dan aku akan lebih tidak perduli' 'Aku mau hidup seribu tahun lagi' '{----------}' '- Chairil Anwar, Maret 1943'	tokenized-input-7.txt test > milestone-1 > tokenized-input-7.txt string ('(* AKU *)') string ('{----------)') string ('Kalau sampai waktuku') string (''Ku mau tak seorang 'kan merayu') string ('Tidak juga kau') string ('Tak perlu sedu sedan itu') string ('Aku ini binatang jalang') string ('Dari kumpulannya terbuang') string ('Biar peluru menembus kulitku') string ('Aku tetap meradang menerjang') string ('Luka dan bisa kubawa berlari') string ('Berlari') string ('Hingga hilang pedih peri') string ('Dan aku akan lebih tidak perduli') string ('Aku mau hidup seribu tahun lagi') string ('{----------}') string ('- Chairil Anwar, Maret 1943')

8.	input-8.txt test > milestone-1 > input-8.txt ` ~ ! @ # $ % ^ & * ( ) - _ = + [ ] \ | ; : ' " , . < > / ?	tokenized-input-8.txt test > milestone-1 > tokenized-input-8.txt unknown (`) unknown (~) unknown (!) unknown (@) unknown (#) unknown ($) unknown (%) unknown (^) unknown (&) times lparent rparent minus unknown (_) unknown (=) plus lbrack rbrack unknown (\) unknown (|) semicolon colon unknown (') unknown (") comma period lss gtr rdiv unknown (?)

Kesimpulan dan Saran

Langkah pertama dalam pembuatan *compiler* untuk bahasa pemrograman Arion adalah pembuatan *lexer*. Dengan melakukan *tokenization* yang mengubah sumber program menjadi token-token kecil sehingga lebih mudah untuk dianalisis alurnya pada langkah berikutnya. Pembuatan *lexer* ini juga merupakan upaya preventif dalam langkah selanjutnya karena kesalahan dalam pengetikan dalam sumber program juga dapat dideteksi lebih awal.

Beberapa saran yang dapat dikembangkan:

- Langkah pertama, pembuatan *lexer*, merupakan langkah paling krusial dalam pembuatan *compiler*. *Lexer* dengan struktur dan logika yang baik dapat memudahkan dalam perancangan sub-sistem lain dalam *compiler* nantinya. Disarankan dalam pembuatan *lexer* ini, logika program compiler yang akan dibangun sudah disusun dengan baik dan matang.
- Membuat prosedur untuk menggabungkan DFA yang berbeda. Hal ini supaya DFA dapat dirancang dalam bentuk yang lebih kecil dan lebih terspesialisasi.

Lampiran

Link repository Github: https://github.com/NafisKreatif/RGX-Tubes-IF2224-2026

Link Workspace Diagram Transisi DFA:
https://drive.google.com/file/d/1t5wUXHwhhgU2Ycq_o6Vizkzg4FSs2KWW/view?usp=sharing

Tabel Pembagian Tugas

No.	Nama	NIM	Pembagian Tugas	Kontribusi (%)
1	Wafiq Hibban Robbany	13524016	Membuat Diagram DFA	23
2	Muhammad Nafis Habibi	13524018	Membuat Class DFA	25
3	An-Dafa Anza Avansyah	13524038	Membuat Diagram DFA	24
4	Wildan Abdurrahman Ghazali	13524054	Membuat Class Tokenizer (Lexer)	28